

# ROUND 2 TECHNICAL INTERVIEW: COMPREHENSIVE STUDY GUIDE

## Candidate Master Defense: Face Recognition System + Code Nimbus AI/ML Intern JD

Company: Code Nimbus Solutions | Role: AI/ML Intern | Target Batch: 2027 | Format: On-Campus F2F

### HOW TO USE THIS MASTER DEFENSE GUIDE IN YOUR CAMPUS INTERVIEW

This guide maps your Face Recognition project directly onto the JD requirements: FastAPI backend scaling, Gemini AI ecosystem (google-gemini SDK), RAG with SentenceTransformers & CrossEncoder reranking, multimodal vision runtimes (CLIP, MediaPipe, OpenCV, ONNX), and observability (Prometheus/OpenTelemetry). Study the bold mathematical terms and code snippets to give confident, production-grade answers.

## PILLAR 1: FACE RECOGNITION PROJECT ARCHITECTURE & PIPELINE

*End-to-end design, detection, alignment, embeddings, and unknown rejection*

### Q1: Can you give a 90-second elevator pitch of your project?

CRITICAL (MUST-KNOW)

- **The Pitch:** I built an end-to-end, CPU-optimized Open-Set Face Recognition and Attendance System in Python with \$0 cloud cost. It solves the 1:N identification problem where an incoming person could either be one of N enrolled users or an un-enrolled stranger. The architecture combines RetinaFace for face detection and 5-point landmark affine alignment, ArcFace (ResNet-50) executed via ONNX Runtime to produce 512-dimensional L2-normalized embeddings, and cosine similarity matching against a persistent template gallery. It features an empirically tuned operating threshold ( $\tau = 0.34$ ) achieving zero False Acceptance on LFW with strict zero data leakage, and includes a full-stack glassmorphic dashboard with live HTML5 webcam attendance and dynamic threshold control.

#### INTERVIEW DEFENSE TIP: FIRST IMPRESSION

Lead with the engineering problem: 'Open-set recognition with unknown rejection'. Mentioning zero data leakage and ONNX CPU optimization immediately separates you from generic tutorial projects.

### Q2: Walk me through the exact pipeline from a camera frame to an ID decision.

CRITICAL (MUST-KNOW)

- **Step 1 (Ingestion & Quality Filter):** Raw BGR frame is captured via OpenCV. Bounding boxes smaller than 30px or detection confidence  $< 0.50$  are filtered out.
- **Step 2 (RetinaFace Detection & Alignment):** Detects 5 facial landmarks (two eyes, nose tip, two mouth corners). Uses an Affine Transformation to warp and normalize the face to a standardized 112x112 frontal canonical view.
- **Step 3 (ArcFace Feature Extraction):** The 112x112 crop passes through an ArcFace ResNet-50 ONNX model, outputting a continuous 512-D latent vector.
- **Step 4 (L2 Normalization):** The raw 512-D vector is normalized to unit length:  $e = v / ||v||_2$ . This guarantees  $||e||_2 = 1.0$ .
- **Step 5 (Cosine Similarity Search):** Computes dot product  $q \cdot e_i$  against enrolled gallery templates. Since both are unit vectors, Cosine Similarity simplifies directly to Dot Product.
- **Step 6 (Unknown Rejection):** If max similarity score  $s \geq 0.34$ , assign person identity. If  $s < 0.34$ , safely reject as 'UNKNOWN'.

### Q3: Detection vs Verification vs Closed-Set vs Open-Set Identification?

HIGH FREQUENCY

- **Face Detection:** Localizing face bounding boxes and facial landmarks in an image  $[x, y, w, h]$ . No identity classification.
- **Face Verification (1:1):** One-to-one claim verification: 'Is this user Alice?' Compares query against Alice's template. Used in smartphone unlock.
- **Closed-Set Identification (1:N):** Assumes the query person is ALWAYS enrolled in the database. Simply returns  $\text{argmax}(\text{sim})$ . Flawed for security because an intruder is always misclassified as the nearest enrolled person.
- **Open-Set Identification (1:N + Rejection):** Our system's task. Solves both: 'Is this person enrolled?' (Thresholding) AND 'If yes, who are they?' (Argmax). Essential for real-world attendance and surveillance.

**Q4: How do you handle edge cases (no face, multiple faces, blur)?**

HIGH FREQUENCY

- **No Face Detected:** RetinaFace returns empty detection list; system gracefully yields 'NO\_FACE\_DETECTED' without crashing, skipping heavy neural inference.
- **Multiple Faces in Frame:** During Enrollment, the system automatically isolates the largest face (by bounding box area  $w * h$ ) and logs a warning. During Identification, the system iterates over ALL detected faces independently and renders green boxes for known users and red boxes for unknown visitors.
- **Motion Blur & Low Light:** Mitigated by calculating the Laplacian Variance of the face crop (`cv2.Laplacian(img, cv2.CV_64F).var()`). Frames with variance  $< 100$  are rejected as blurry before running ArcFace.

**PILLAR 2: MATHEMATICS OF EMBEDDINGS & ARCFACE LOSS***Angular margin loss, hyperspherical geometry, and metric properties***Q5: Why ArcFace? How does Additive Angular Margin Loss work mathematically?**

CRITICAL (MUST-KNOW)

Traditional Softmax Loss separates classes using hyperplanes in Euclidean space, but does not enforce compact intra-class clustering. Triplet Loss minimizes anchor-positive distance while maximizing anchor-negative distance, but suffers from combinatorial explosion and unstable semi-hard negative mining.

## &gt;&gt; The ArcFace Formulation

ArcFace reformulates Softmax on a normalized hypersphere. Feature vectors  $x_i$  and weight vectors  $W_j$  are normalized ( $\|x_i\|=1, \|W_j\|=1$ ), making  $W_j^T x_i = \cos(\theta_{ij})$ . It then adds an Additive Angular Margin  $m$  ( $m = 0.5$  rad) directly to the target ground-truth angle  $\theta_{yi}$ :

```
L_ArcFace = -log( exp(s * cos(theta_yi + m)) /  
                (exp(s * cos(theta_yi + m)) + sum_{j != yi} exp(s * cos(theta_j))) )  
where s = feature scale parameter (typically 64.0), m = angular margin (0.5)
```

- **Intuition:** Because cosine is strictly monotonically decreasing on  $[0, \pi]$ , adding margin  $m$  penalizes the target angle, forcing the network during backpropagation to compress embeddings of the same identity into an extremely tight geodesic cone on the unit sphere.

**Q6: Why Cosine Similarity over Euclidean Distance? Why Dot Product?**

CRITICAL (MUST-KNOW)

- **Direction vs Magnitude:** ArcFace maps identity to angular direction on the hypersphere. Vector magnitude often correlates with image brightness or contrast rather than identity. Cosine similarity evaluates pure angular direction:  $\cos(\theta) = (A \cdot B) / (\|A\| * \|B\|)$ .
- **Mathematical Equivalence:** Because all vectors in our pipeline are strictly L2-normalized ( $\|q\|_2 = 1.0, \|e\|_2 = 1.0$ ):

```
Cosine_Similarity(q, e) = (q · e) / (1.0 * 1.0) = q · e = sum_{k=1}^{512} q_k * e_k
```

- **Production Speedup:** Computing dot products avoids square roots and divisions, enabling 1:N search to run as a single BLAS-optimized matrix multiplication ( $q @ E^T$ ) across thousands of templates in  $<1ms$ .

**Q7: What is max\_similarity vs mean\_prototype matching?**

HIGH FREQUENCY

- **max\_similarity (Default):** Compares query vector  $q$  against ALL enrolled photo vectors  $\{e_1, e_2, \dots\}$  for person  $P$  and takes  $\max(q \cdot e_i)$ . Preserves multi-modal facial variation (e.g. smiling vs neutral, slight angle changes).
- **mean\_prototype:** Calculates a single centroid vector  $u_P = \text{normalize}(\text{mean}(e_1, e_2, \dots))$ . Faster storage, but can wash out extreme variations.

**PILLAR 3: EVALUATION METRICS, THRESHOLD TUNING & SCALE***FAR, FRR, EER, zero-leakage protocol, and the 1:N Extreme Value problem***Q8: Define FAR, FRR, and EER. How did you pick  $\tau = 0.34$ ?**

CRITICAL (MUST-KNOW)

- **False Acceptance Rate (FAR):**  $\text{FAR} = \text{False Accepts} / \text{Total Impostor Queries}$ . The rate at which an unauthorized stranger is mistakenly accepted as an enrolled user. (Security Risk).
- **False Rejection Rate (FRR):**  $\text{FRR} = \text{False Rejects} / \text{Total Genuine Queries}$ . The rate at which an authorized enrolled user is rejected as unknown. (Convenience Issue).
- **Equal Error Rate (EER):** The threshold where FAR equals FRR. Represents the intrinsic discriminative capacity of the model.

- **Zero-Leakage Tuning:** Threshold was evaluated across 81 increments (0.10 to 0.90) STRICTLY on the validation split (80 known, 80 unknown). We selected  $\tau = 0.34$  by optimizing: Maximize Known Top-1 Accuracy subject to  $\text{FAR} \leq 1.0\%$ . On validation,  $\tau = 0.34$  gave  $\text{FAR} = 0.0\%$  and 100% accuracy. EER occurred at  $\tau = 0.31$ .

**Q9: Your test set got 100% accuracy. Is that realistic in production? Why does it degrade?**

CRITICAL (MUST-KNOW)

- **Honest Engineering Answer:** No, 100% test accuracy is an artifact of a small benchmark gallery ( $N = 40$  people) and ArcFace's huge angular margin on LFW (genuine pairs score 0.55-0.85, while impostor pairs stay  $< 0.18$ ).
- **The 1:N Extreme Value Problem:** In a real enterprise with  $N = 100,000$  enrolled identities, an unknown visitor is compared against 100,000 random vectors. By Extreme Value Theory, the distribution of  $\max_{i=1..N} (q \cdot e_i)$  shifts significantly to the right. The probability of an accidental random alignment  $\geq 0.34$  increases drastically.
- **Production Mitigations:** 1. Elevate threshold ( $\tau \geq 0.50$ ). 2. Score Normalization (T-norm / Z-norm). 3. Multi-image verification and active liveness detection.

**PILLAR 4: FASTAPI BACKEND ARCHITECTURE & SCALING***Direct JD Requirement: High-performance ML APIs, async vs CPU threadpools, Pydantic***Q10: How would you architect this Face Recognition System as a FastAPI service?**

DIRECT JD MATCH

- **Modular Router Structure:** Separate endpoints into `api/v1/enroll.py`, `api/v1/identify.py`, and `api/v1/attendance.py` using FastAPI `APIRouter`.
- **Lifespan Context Manager:** Load heavy ONNX models once during server startup using `@asynccontextmanager`, storing the embedder and vector index in `app.state`.
- **Pydantic Validation:** Define strict request/response schemas for payload validation and automatic OpenAPI Swagger documentation.

```
from fastapi import FastAPI, UploadFile, File, Depends, HTTPException
from pydantic import BaseModel, Field
from typing import List, Optional

class IdentifyResponse(BaseModel):
    name: str
    score: float = Field(..., ge=0.0, le=1.0)
    is_known: bool
    bbox: List[int]
    timestamp: str
```

**Q11: In FastAPI, should /identify be 'async def' or 'def'? Explain Event Loop blocking.**

CRITICAL (MUST-KNOW)

THIS IS THE #1 FASTAPI INTERVIEW TRAP. Interviewers test if you understand the Python GIL and AsyncIO Event Loop.

- **The Trap:** If you write `'async def identify(file: UploadFile):'` and run CPU-heavy operations inside (OpenCV decode, RetinaFace ONNX inference, matrix multiplication), the synchronous CPU work BLOCKS the single-threaded asyncio event loop. No other request can be accepted or processed while that face is running!
- **The Solution (Option A - Standard def):** Define the route with standard `'def identify(...)'`. FastAPI automatically executes standard `'def'` endpoints in an external threadpool (anyio worker threads), keeping the event loop 100% non-blocking.
- **The Solution (Option B - Explicit Worker Pool):** If using `'async def'`, offload the CPU-bound inference explicitly to a worker threadpool using `asyncio.to_thread`:

```
@app.post('/api/v1/identify', response_model=IdentifyResponse)
async def identify_face(file: UploadFile = File(...)):
    contents = await file.read() # Asynchronous I/O read
    # Offload CPU-heavy inference to threadpool without blocking event loop:
    result = await asyncio.to_thread(embedder.process_and_match, contents)
    return result
```

**Q12: Base64 JSON vs Multipart/form-data (UploadFile) in ML APIs?**

HIGH FREQUENCY

- **Base64 JSON:** Pros: Easy to transmit over WebSockets or single JSON payloads from frontend HTML5 canvases. Cons: Base64 encoding inflates payload size by ~33% and consumes CPU to decode strings back to byte buffers.
- **UploadFile (Multipart):** Pros: Streams raw binary bytes directly into `SpooledTemporaryFile` with zero encoding overhead; highly efficient for multi-megapixel images and batch enrollments.

- **Best Practice:** Use Base64 for live webcam streaming frames (<100KB); use UploadFile for batch enrollment photos.

## PILLAR 5: GEMINI AI ECOSYSTEM & MULTIMODAL GENAI

*Direct JD Requirement: google-genai SDK, multimodal reasoning, and structured outputs*

### Q13: What is the new google-genai SDK, and how does it differ from older libraries?

DIRECT JD MATCH

- **google-genai SDK:** Google's modern, official unified Python SDK for Gemini 1.5 Flash, 1.5 Pro, and 2.0 models. It replaces legacy packages with a cleaner client-centric architecture.
- **Key Enhancements:** 1. Single unified Client (client = genai.Client()). 2. Native Pydantic structured output validation (response\_schema). 3. Native multimodal support (images, video, audio, PDFs). 4. System instructions, thinking models, and built-in function calling.

### Q14: How would you integrate Gemini into your Face Recognition & Attendance System?

DIRECT JD MATCH

- **Use Case 1 (Multimodal ID Card KYC Verification):** During employee/student enrollment, capture a webcam selfie and a photo of their physical ID card. Feed both images to Gemini 1.5 Flash to verify facial match, extract text (Name, Roll No, Department), and detect fake/tampered cards.
- **Use Case 2 (Intruder Incident Auto-Reporting):** When an 'UNKNOWN' intruder is detected during off-hours, pass the video clip/frames to Gemini to generate an automated incident narrative: 'Unknown male subject in dark hoodie entered through corridor B at 11:42 PM; facial features matched no registered personnel.'
- **Use Case 3 (Visual Demographics & Emotion Analytics):** Use Gemini's multimodal reasoning to extract auxiliary non-biometric attributes (glasses, mask, approximate mood/fatigue) for workplace wellness analytics.

### Q15: Write a Python snippet using google-genai SDK for multimodal analysis with Pydantic.

DIRECT JD MATCH

```
from google import genai
from google.genai import types
from pydantic import BaseModel
from PIL import Image

class IDVerificationResult(BaseModel):
    id_card_name: str
    faces_match: bool
    confidence_score: float
    tampering_detected: bool
    notes: str

client = genai.Client(api_key='YOUR_GEMINI_API_KEY')
response = client.models.generate_content(
    model='gemini-2.0-flash',
    contents=[Image.open('selfie.jpg'), Image.open('student_id.jpg'),
              'Verify if the person in the selfie matches the student ID card.'],
    config=types.GenerateContentConfig(
        response_mime_type='application/json',
        response_schema=IDVerificationResult,
        system_instruction='You are an enterprise biometric security compliance auditor.'
    )
)
result: IDVerificationResult = response.parsed
```

## PILLAR 6: SEMANTIC SEARCH & RAG PIPELINES

*Direct JD Requirement: SentenceTransformers, CrossEncoder reranking, and vector search*

**Q16: How do Face Embeddings connect to Text Embeddings in Semantic Search?**

DIRECT JD MATCH

- **Conceptual Equivalence:** Both map unstructured raw inputs into a dense vector space (hypersphere) where semantic meaning or biological identity equals geometric distance.
- **Face Recognition:** Raw face pixels -> ArcFace (ResNet-50) -> 512-D vector -> Cosine similarity.
- **Semantic Search:** Raw text sentence -> SentenceTransformer (BERT/RoBERTa) -> 384-D / 768-D vector -> Cosine similarity.
- **Indexing:** Both systems perform 1:N nearest neighbor retrieval against stored vector indices (FAISS, Chroma, Qdrant).

**Q17: Bi-Encoder (SentenceTransformers) vs Cross-Encoder (Reranker)? Why use both?**

CRITICAL (MUST-KNOW)

- **Bi-Encoder (SentenceTransformers):** Encodes Query  $q$  and Document  $d$  independently into vectors  $u$  and  $v$ . Similarity is simply  $u \cdot v$ . Documents can be pre-embedded offline and stored in a vector index. Extremely fast: retrieves Top-50 candidates from millions of docs in <10ms. Drawback: No cross-token attention between query and document words.
- **Cross-Encoder (CrossEncoder Reranker):** Passes Query and Document TOGETHER into full self-attention layers: [CLS] Query [SEP] Document. Captures rich token-level interaction, yielding vastly higher ranking accuracy. Drawback: Cannot pre-compute embeddings; requires running heavy transformer forward passes at query time. Too slow for full-database search.
- **The Two-Stage Pipeline (Industry Standard RAG):** Stage 1 (Retrieval): Bi-Encoder + Vector DB quickly retrieves top 50 candidates in <10ms. Stage 2 (Reranking): CrossEncoder scores the top 50 to select the top 5 most relevant chunks in <40ms. This achieves the speed of a Bi-Encoder with the precision of a Cross-Encoder.

```
from sentence_transformers import SentenceTransformer, CrossEncoder

# Stage 1: Fast Bi-Encoder Retrieval (ANN Vector Search)
bi_encoder = SentenceTransformer('all-MiniLM-L6-v2')
query_vector = bi_encoder.encode('What is the attendance policy?')
top_50_docs = vector_db.search(query_vector, top_k=50)

# Stage 2: High-Precision Cross-Encoder Reranking Filter
reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')
pairs = [[query, doc.text] for doc in top_50_docs]
scores = reranker.predict(pairs)
top_5_docs = [doc for _, doc in sorted(zip(scores, top_50_docs), reverse=True)[:5]]
```

**Q18: How would you build an Enterprise RAG pipeline from scratch?**

HIGH FREQUENCY

- **1. Document Ingestion & Chunking:** Parse documents (PDF, Markdown). Apply recursive token-aware chunking (chunk size: 512 tokens, overlap: 50 tokens) to preserve semantic context across chunk boundaries.
- **2. Dense Vector Indexing:** Compute embeddings with SentenceTransformers ('all-mpnet-base-v2') and index into a vector DB with HNSW indexing.
- **3. Hybrid Search:** Combine BM25 keyword search (sparse) with dense vector search to capture both exact domain terms and semantic intent via Reciprocal Rank Fusion (RRF).
- **4. Cross-Encoder Reranking:** Filter candidate pool down to top 3-5 high-relevance chunks using a CrossEncoder.
- **5. Context Assembly & LLM Generation:** Inject reranked chunks into Gemini 1.5 Flash with strict grounding prompt: 'Answer using only the provided context. If unknown, state unverified.'

**PILLAR 7: VISION FRAMEWORKS, EDGE RUNTIMES & OBSERVABILITY***CLIP, MediaPipe, OpenCV, ONNX Runtime, Prometheus, and OpenTelemetry***Q19: What is CLIP and how does Multimodal Contrastive Learning work?**

DIRECT JD MATCH

- **CLIP (Contrastive Language-Image Pretraining):** OpenAI model featuring two parallel encoders: a Vision Transformer (ViT) for images and a Text Transformer for text captions.
- **Shared Embedding Space:** Trained using InfoNCE loss on 400M (image, text) pairs. It projects both image vectors and text vectors into the SAME shared metric hypersphere.
- **Multimodal Search Application:** Enables zero-shot text-to-image queries: To find 'a person wearing a medical mask', embed the text with CLIP text encoder, compute cosine similarity against image embeddings, and retrieve matching frames.

**Q20: Why ONNX Runtime? How does it optimize CPU inference in your project?**

DIRECT JD MATCH

- **Graph Optimizations:** Merges redundant layers (Operator Fusion: Conv + BatchNorm + ReLU merged into a single optimized fused kernel). Eliminates dead computation nodes.
- **Multi-Threaded Parallelism:** ONNX Runtime leverages OpenMP and intra-op thread allocation to saturate all CPU cores during ArcFace inference, dropping latency to ~150ms per face on commodity laptops.
- **INT8 Quantization:** Converts 32-bit floating point weights to 8-bit integers (Post-Training Quantization). Reduces model memory footprint by 75% (from 250MB to ~60MB) and doubles CPU throughput with <1% accuracy loss.

**Q21: MediaPipe vs OpenCV vs RetinaFace: When do you use each?**

DIRECT JD MATCH

- **OpenCV:** General-purpose computer vision library: image I/O, resizing, colorspace conversion (BGR to RGB), geometric affine transformations, matrix drawing.
- **MediaPipe:** Ultra-lightweight edge framework by Google. Ideal for real-time mobile/browser tasks (BlazeFace runs at 100+ FPS, 468-point 3D Face Mesh, hand landmark tracking). Best for client-side liveness detection.
- **RetinaFace:** Heavy deep learning detector with Feature Pyramid Networks (FPN). Slower than MediaPipe BlazeFace, but significantly higher accuracy on tiny faces, extreme occlusion, and poor lighting.

**Q22: How would you monitor this API with Prometheus, Grafana, and OpenTelemetry?**

DIRECT JD MATCH

- **Prometheus (Metrics):** Instrument FastAPI with prometheus-fastapi-instrumentator to expose a /metrics endpoint. Track: 1. Histogram: 'http\_request\_duration\_seconds' (monitor p95/p99 latency). 2. Counter: 'face\_identification\_total{status="matched|unknown|no\_face"}'. 3. Gauge: 'enrolled\_gallery\_size'.
- **Grafana (Visualization):** Build dashboards displaying real-time FPS throughput, inference latency percentiles, error rate spikes, and CPU/RAM memory utilization.
- **OpenTelemetry (Distributed Tracing):** Add trace spans to decompose end-to-end request latency: Image Decode Span (5ms) -> RetinaFace Detection Span (90ms) -> ArcFace Embedding Span (120ms) -> Vector Search Span (2ms). If p95 latency spikes, trace spans pinpoint the exact bottleneck.

**PILLAR 8: PRODUCTION SCALING TO 100,000+ IDENTITIES & ANTI-SPOOFING***Vector databases (FAISS, Milvus), HNSW, and presentation attack mitigation***Q23: How would you scale your template database from 40 to 100,000 people?**

CRITICAL (MUST-KNOW)

- **The Bottleneck:** Linear scan (NumPy matrix dot product) takes  $O(N * D)$ . For  $N = 100,000$  templates, computing 100k dot products per incoming camera frame causes high CPU latency and stalls.
- **Solution 1 (FAISS Vector Indexing):** Replace linear search with FAISS (Facebook AI Similarity Search) using HNSW (Hierarchical Navigable Small World) graphs or IVF-PQ (Inverted File with Product Quantization). Search complexity drops from  $O(N)$  to  $O(\log N)$ , returning nearest neighbors in <3ms.
- **Solution 2 (Vector Database):** Deploy Milvus, Qdrant, or Pgvector. Decouples vector indexing and filtering from the web server with horizontal scalability.
- **Solution 3 (Microservices Architecture):** Decouple into an asynchronous ingestion queue (RabbitMQ/Kafka) -> GPU Face Worker Pods (for RetinaFace + ArcFace) -> Vector Search Service (gRPC).

**Q24: How do you protect the system against Presentation Attacks (Spoofing)?**

HIGH FREQUENCY

- **The Attack:** An adversary holds up a printed color photograph, tablet video, or 3D silicone mask of an enrolled employee to spoof attendance.
- **Passive Anti-Spoofing:** Deploy a lightweight CNN (e.g. MiniFASNet) to analyze high-frequency Fourier spectrum, specular screen reflections, and moire interference patterns typical of digital screens.
- **Active Liveness:** Prompt the user with dynamic real-time challenges: 'Blink twice', 'Turn head 30 degrees right', or track eye pupil dilation against a random screen color flash.
- **Multi-Spectral Hardware:** Integrate Near-Infrared (NIR) or 3D Time-of-Flight (ToF) depth cameras; printed photos and phone screens appear completely black or flat under infrared illumination.

## PILLAR 9: ROUND 2 CAMPUS INTERVIEW STRATEGY & BEHAVIORAL SCRIPT

*The 60-second intro, answering trade-offs, and high-impact questions to ask*

### Q25: The 60-Second Self-Introduction & Project Pitch

CRITICAL (MUST-KNOW)

"Hi, I'm Karthik, a 2027 graduate passionate about production AI engineering and backend systems. In my recent project, I built a production-grade Face Recognition and Attendance platform optimized for zero cloud cost and CPU execution. I tackled core machine learning challenges like open-set recognition with unknown rejection using ArcFace 512-dimensional embeddings and RetinaFace alignment. I ensured zero data leakage with a rigorous train/val/test protocol on LFW and tuned the decision threshold empirically. Beyond Computer Vision, I'm deeply interested in high-performance ML backends with FastAPI, modern GenAI workflows using the google-genai SDK, and RAG architectures combining SentenceTransformers with CrossEncoder reranking. I'm really excited about the AI/ML Intern role at Code Nimbus Solutions because it aligns directly with what I love building: deploying scalable, production-ready AI pipelines that deliver real business impact."

### Q26: Top 3 Thoughtful Questions to Ask the Interviewer

HIGH FREQUENCY

- **Question 1:** "In Code Nimbus's production RAG workflows, what reranking strategy or CrossEncoder models have you found strike the best balance between latency and retrieval accuracy?"
- **Question 2:** "How does the team currently handle observability and trace latency across multimodal models—do you use OpenTelemetry and Prometheus, and what are your p95 latency targets?"
- **Question 3:** "For an intern joining the AI/ML team, what does a successful first month look like in terms of shipping models or FastAPI microservices to production?"